**UNIVERSITE GASTON BERGER DE SAINT-LOUIS**




**UFR DES SCIENCES APPLIAQUÉES ET TECHNOLOGIES**

**SECTION INFORMATIQUE**

**NIVEAU MASTER II**

# Base de données Distribuées

**Présenté par**

Kouna Samb P32 300

Aby Mbaye P32 864

Ahmadou Bamba Mboup P32 379

**Sous la direction de**

M. Gaye

**Année Académique : 2025/2026**

## 1. Contexte, problématique et objectifs

Une plateforme de commerce électronique internationale doit traiter des demandes concurrentes venant de régions différentes. Elle doit continuer à fonctionner lorsqu'un nœud devient indisponible et empêcher que plusieurs clients achètent les mêmes dernières unités.

Problématique : comment répartir les données et les traitements sur plusieurs nœuds tout en conservant la cohérence des commandes et des stocks ?

## 2. Objectifs

Répartir les données et les accès entre quatre nœuds logiques correspondant à quatre régions.

Exposer les opérations métier au moyen d'une API REST.

Empêcher la survente lors d'achats simultanés.

Évaluer le comportement du système pendant une panne et sous charge.

Vérifier les invariants à partir d'un audit de la base.

## 3. Choix théoriques et justifications techniques

### 3.1 Choix de CockroachDB

CockroachDB est une base SQL distribuée compatible avec le protocole PostgreSQL. Elle fournit des mécanismes intégrés de partitionnement en plages, de réplication et de transactions distribuées. Ce choix évite de réimplémenter un protocole de consensus et la coordination transactionnelle dans un middleware personnalisé.

### 3.2 Réplication et consensus Raft

La commande de configuration de zone a retourné num_replicas = 3. L'inspection des plages de la table produits a montré, pour les plages affichées, trois réplicas votants répartis entre plusieurs régions. Les réplicas d'une plage contiennent des copies de cette plage, et non nécessairement une copie complète de la base.

CockroachDB utilise Raft pour coordonner les réplicas d'une plage. Avec trois réplicas votants, deux votes constituent normalement la majorité nécessaire. La disponibilité effective dépend toutefois du placement des réplicas et de la capacité des plages concernées à conserver un quorum.

### 3.3 Transactions et cohérence

Le prototype utilise les transactions SQL et le niveau d'isolation SERIALIZABLE pour coordonner les opérations concurrentes. Une commande pouvant toucher plusieurs produits et régions, les modifications liées doivent être traitées de façon atomique.

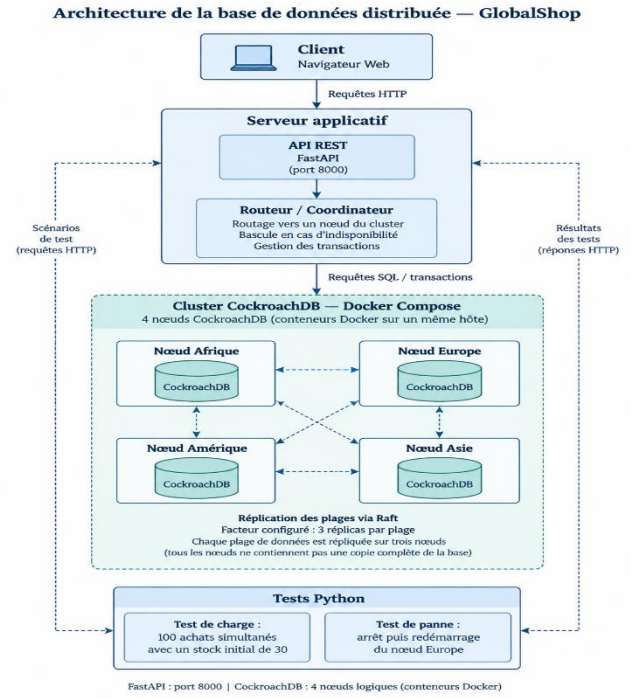
### 3.4 Routeur FastAPI

Le routeur expose les opérations clients, produits et commandes. Il choisit une passerelle SQL selon la région du client et peut tenter une autre passerelle en cas de problème de connexion, selon le code du projet. Le routeur n'étant pas répliqué dans ce prototype, il reste un point potentiel de défaillance unique.

## 4. Architecture du système

L'architecture comprend quatre nœuds CockroachDB :  Afrique, Europe, Amérique et Asie et un routeur FastAPI accessible en HTTP sur le port 8000. Docker Compose orchestre les conteneurs ; un conteneur d'initialisation prépare le schéma et la configuration.

Clients des quatre régions → Routeur FastAPI (port 8000) → Cluster CockroachDB : Afrique, Europe, Amérique, Asie. Réplication des plages via Raft, facteur configuré à 3.




Clients/API : envoient les requêtes HTTP.

Routeur FastAPI : reçoit les requêtes, choisit une passerelle et interroge la base.

Cluster CockroachDB : distribue les plages de données et réplique les plages selon la configuration.

Initialisation SQL : prépare le schéma et les paramètres au démarrage.

Tests Python : vérifient les invariants, la charge et le comportement pendant une panne.

## 5. Stratégie de partitionnement et de réplication

### 5.1 Partitionnement

Le projet utilise une stratégie de découpage par région. Les préfixes régionaux regroupent logiquement les données associées à une région. CockroachDB organise l'espace de clés en plages (*ranges*), qui peuvent être réparties et rééquilibrées entre les nœuds.

Cette stratégie vise à rapprocher les données des utilisateurs, mais une région très sollicitée peut être plus chargée que les autres. Le prototype tourne sur une seule machine physique ; il ne mesure donc pas les latences réelles entre continents.

## 5.2 Réplication observée

La configuration affiche un facteur de réplication de trois. Les plages de la table produits examinées affichent trois réplicas votants, par exemple {1,2,4} ou {1,3,4}. Les groupes de nœuds varient selon les plages.

La réplication porte sur les plages de données : chaque nœud héberge des réplicas de certaines plages, et ne possède pas nécessairement toute la base. La configuration à trois réplicas ne signifie donc pas que chaque ligne est copiée sur les quatre nœuds.

# 6. Prévention de la survente et transactions concurrentes

Le risque métier principal est que deux clients achètent simultanément les dernières unités d'un produit. Le prototype combine plusieurs protections :

Décrément conditionnel : le stock diminue uniquement si la quantité disponible est suffisante.

Transactions : les modifications liées à une commande sont regroupées dans une transaction.

Isolation SERIALIZABLE : les transactions incompatibles peuvent entrer en conflit et nécessiter une nouvelle tentative.

Contraintes d'intégrité : le stock doit rester supérieur ou égal à zéro.

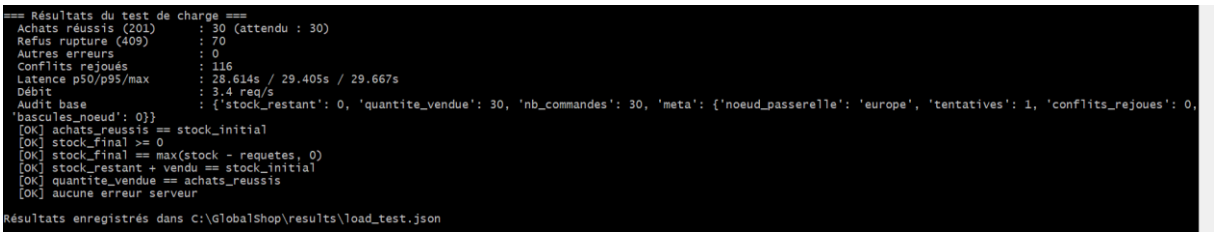
Réponse métier : les achats sans stock suffisant sont refusés, notamment avec le code HTTP 409 dans les scénarios testés.

Une commande portant sur des produits de régions différentes devient une transaction distribuée. La base doit coordonner les changements afin d'éviter qu'une partie de la commande soit validée indépendamment des autres.

# 7. Protocole et résultats des tests

## 7.1 Test de charge

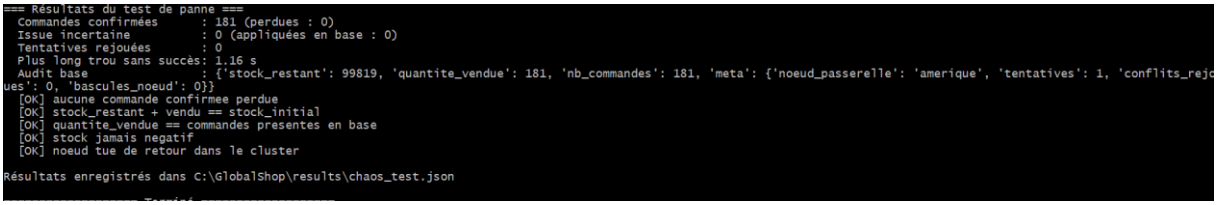
Le test a envoyé 100 achats concurrents sur un produit dont le stock initial était de 30 unités. Les demandes provenaient de clients des quatre régions. Les contrôles portaient sur les achats réussis, les refus, le stock final, les quantités vendues et les erreurs serveur.




Interprétation : exactement 30 achats ont réussi et les 70 demandes excédentaires ont été refusées. Le stock final est nul, la quantité vendue correspond aux achats réussis et aucune autre erreur n'a été comptabilisée. Les conflits rejoués montrent le coût de la contention lorsque de nombreuses transactions modifient le même produit.

## 7.2 Test de panne

Le test a arrêté brutalement le conteneur globalshop-node-europe pendant le traitement des commandes, puis l'a redémarré. L'audit final a comparé le stock restant, la quantité vendue et le nombre de commandes en base.




Interprétation : les quatre nœuds sont revenus à l'état « up ». Le test a confirmé 181 commandes, aucune commande confirmée perdue et aucun stock négatif. Le plus long intervalle sans commande confirmée a été de 1.16 s. Aucune tentative rejouée n'a été comptabilisée dans ce scénario : il ne démontre donc pas, à lui seul, que le mécanisme de rejeu a effectivement été déclenché.

## 8. Analyse de résilience

Les essais fournissent des éléments expérimentaux en faveur de la cohérence des stocks et de la continuité du service pendant l'arrêt temporaire du nœud Europe. Les commandes confirmées sont restées présentes en base et les contrôles du stock ont été satisfaits.

## 9. Conclusion

GlobalShop met en œuvre un prototype de base de données distribuée fondé sur CockroachDB, quatre nœuds logiques et un routeur FastAPI. La configuration de réplication à trois réplicas a été vérifiée et les plages de la table produits examinées affichent trois réplicas votants.

Le test de charge a confirmé 30 achats réussis sur un stock de 30, avec 70 refus et aucune autre erreur. Le test de panne a confirmé 181 commandes, aucune commande confirmée perdue et la cohérence de l'audit du stock. Ces résultats sont positifs pour les scénarios exécutés, tout en laissant des limites à traiter, notamment l'absence de régions physiques distinctes et le point unique de défaillance que représente le routeur.

Le projet illustre l'intérêt d'une base SQL distribuée qui intègre la réplication et les transactions, tout en montrant que la résilience doit être évaluée par des mesures concrètes et dans des conditions représentatives.